

MIDPOINT REPORT

Building World Models for Reinforcement Learning with Spectral Transfer Units

Shivam Kak
Princeton University
Department of Computer Science
sk3686@princeton.edu

Elad Hazan (Advisor)
Princeton University
Department of Computer Science
ehazan@princeton.edu

Abstract—Reinforcement learning agents often suffer from sample inefficiency when learning complex tasks through trial and error. World models offer a solution by enabling agents to simulate future outcomes and practice in imagined rollouts. This thesis explores the use of Spectral Transfer Units (STUs) as a core architecture for world models in discrete and continuous control environments. STUs leverage spectral filtering methods rooted in dynamical systems theory to efficiently capture temporal dependencies and predict long-horizon dynamics. Through experiments with the EfficientZero algorithm on Atari environments, we evaluate various architectural modifications incorporating STUs into the value, policy, and dynamics networks. Our preliminary results show a promising acceleration in training convergence, though challenges remain in preventing overfitting and maintaining stable long-term predictions.

Index Terms—Reinforcement Learning, World Models, Spectral Transfer Units, Spectral Filtering, Dynamical Systems, EfficientZero, Atari

I. MOTIVATION

REINFORCEMENT learning (RL) agents must learn to make decisions in complex environments by trial and error, which can be highly sample-inefficient. However, learning accurate world models for control tasks remains challenging, particularly for long-horizon predictions where small errors compound over time.

This work is important because improved world models could dramatically reduce the sample complexity of RL algorithms, making them more practical for real-world applications like robotics, autonomous systems, and control problems where data collection is expensive or risky.

The goal of this project is to develop and evaluate Spectral Transfer Units (STUs) as the core architecture for world models in discrete and continuous control reinforcement learning environments, with the aim of achieving more accurate long-term dynamics prediction than existing approaches.

II. BACKGROUND AND RELATED WORK

A. Limitations of Existing World Model Architectures

Existing world model architectures face several limitations. Recurrent neural networks suffer from vanishing gradients and struggle with long sequences. Transformer-based models, while powerful, require substantial computational resources and may not capture the underlying dynamical structure of physical systems efficiently. Most critically, prediction errors in neural network-based world models tend to compound

rapidly over time, limiting their utility for long-horizon planning.

B. Spectral Transfer Units

STUs offer a fundamentally different approach rooted in spectral methods for learning dynamical systems. Originally developed for linear dynamical systems [1] and recently extended to universal nonlinear dynamics, STUs use spectral filtering to efficiently capture temporal dependencies. Their theoretical foundation in dynamical systems theory makes them particularly well-suited for modeling the transition dynamics that world models must learn.

C. EfficientZero Algorithm

The EfficientZero algorithm [2] learns a predictive model in a latent space and then performs planning over actions using this model. The components of EfficientZero consist of:

- **Representation Function:** $H : s_t = H(o_t)$ — learns a compact state representation of the input observation o_t
- **Dynamic Function:** $G : \hat{s}_{t+1}, r_t = G(s_t, a_t)$ — predicts the next state $\hat{s}_{t+1}$ and the reward r_t
- **Policy Function:** $P : p_t = P(s_t)$ — outputs the current policy p_t
- **Value Function:** $V : v_t = V(s_t)$ — provides the value estimation v_t at the current state

All components are implemented as neural networks. The action embedding block A encodes the action a_t as a latent vector within the dynamic function.

D. Spectral Filtering Theory

A Spectral Transform Unit (STU) is a sequence-to-sequence neural network layer [3]. Given an input sequence $\{u_1, \dots, u_L\} \in \mathbb{R}^{d_{in}}$, it produces an output sequence $\{y_1, \dots, y_L\} \in \mathbb{R}^{d_{out}}$. A single layer of STU is parameterized by a number K denoting the number of eigenfactors, and matrices $M_1^{\phi+}, \dots, M_K^{\phi+}, M_1^{\phi-}, \dots, M_K^{\phi-} \in \mathbb{R}^{d_{out} \times d_{in}}$, and $M_1^u, M_2^u, M_3^u \in \mathbb{R}^{d_{out} \times d_{in}}$.

The Hankel matrix $Z \in \mathbb{R}^{L \times L}$ whose entries are given by:

$$Z[i, j] \triangleq \frac{2}{(i+j)^3 - (i+j)} \quad (1)$$

Let $\{(\sigma_j \in \mathbb{R}, \phi_j \in \mathbb{R}^L)\}_{j=1}^L$ be the eigenvalue-eigenvector pairs of Z ordered to satisfy $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d$.

III. PROPOSED APPROACH/METHODOLOGY

Our approach involves integrating Spectral Transfer Units into the EfficientZero world model architecture at various points to evaluate their effectiveness in learning dynamics. We focus on three primary integration points:

- 1) **Value/Policy Networks:** Adding STU layers to the value and policy prediction heads to improve state evaluation
- 2) **Dynamics Network:** Incorporating STU layers in the dynamics function to better capture temporal dependencies in state transitions
- 3) **History Tracking:** Implementing a history buffer to provide real sequences of input states to STU layers rather than flattened neural network features

We evaluate these modifications on the Atari Pong environment, using training loss, policy loss, and evaluation scores as our primary metrics. Each experiment runs for approximately 100,000 training steps (roughly 15 hours on an A100 GPU).

IV. PROGRESS TO DATE

A. Initial Setup and Literature Review

We began this project with a comprehensive paper review of existing literature on spectral filtering and world models. We then worked on creating a PyTorch port of the Deluca codebase (originally in Jax), which colleagues in the lab had been using for spectral filtering experiments on simple RL environments. This port enables more diverse future experiments, as PyTorch is more widely adopted for testing many RL environments.

Following this preliminary work, we focused on applying spectral filtering to the EfficientZero algorithm for Atari environments, starting with the Pong environment as the simplest discrete action space testbed. The EfficientZero library, based on the earlier MuZero architecture [4], required updating outdated dependencies to run on A100 GPUs.

B. Experimental Results

We conducted a series of experiments to evaluate different STU integration strategies. Due to the computational cost (approximately 15 hours per 100K-step experiment), we ran single instances of each experimental condition, with the plan to run multiple replicates for promising configurations to establish statistical significance.

1) *Experiment 1: STU in Value Prediction:* In the first experiment, we added a MiniSTU layer to the value prediction function within the ValuePolicy prediction network, utilizing the MiniSTU module developed by peers in Professor Hazan’s lab.

As shown in Figure 1, the STU version achieved a total loss of approximately -18.2 roughly $8.3\times$ faster than the benchmark version (STU: 13,000 steps vs. benchmark: 108,000 steps). However, the STU version subsequently overcorrected, and the final evaluation score did not beat the benchmark. This promising acceleration motivated further hyperparameter exploration.

2) *Experiment 2: STU in Both Value and Policy Prediction:* We extended the STU integration to both the value and policy prediction heads on top of the existing EfficientZero structure. The results in Figure 2 showed similar patterns to Experiment 1, with rapid initial convergence but subsequent instability.

3) *Experiments 3-5: Hyperparameter Tuning:* To address the overfitting observed in Experiments 1 and 2, we conducted three hyperparameter tuning experiments, shown in Figure 3:

- **Experiment 3** (Figure 3(a)): Reduced learning rate by a factor of 10 to prevent potential overfitting
- **Experiment 4** (Figure 3(b)): Increased weight decay by a factor of 5 to curb overfitting
- **Experiment 5** (Figure 3(c)): Increased the number of filters and sequence length (from 8 to 16 filters) to handle potentially larger input states

None of these hyperparameter modifications resulted in performance exceeding the original baseline.

4) *Experiments 6-7: STU in Dynamics Network:* Given the limited success with value/policy network modifications, we shifted focus to the dynamics network, which directly predicts state transitions.

Experiment 6 (Figure 4): We added STU to the dynamics network with a residual connection:

$$\text{output}_{\text{new}} = \text{MLP}(\text{output}_{\text{old}} + \text{MiniSTU}(\text{output}_{\text{old}})) \quad (2)$$

Experiment 7 (Figure 5): We implemented a HistoryMiniSTU component that maintains a history buffer of input states to the dynamics network. The goal was to provide real sequences of states to the STU block rather than flattened features from a single state. While this approach showed some changes in policy loss, the total loss remained greater than the benchmark, and the training loop terminated early, indicating problems with the history buffer implementation.

V. CURRENT CHALLENGES

Several key challenges have emerged from the experimental work:

- 1) **Overfitting and Instability:** The most promising results (Experiment 1) show rapid initial convergence followed by overcorrection, suggesting that the STU-augmented models may be overfitting to early training data. Standard regularization techniques (reduced learning rate, increased weight decay) have not resolved this issue.
- 2) **Lack of Dynamics-Specific Loss:** The EfficientZero library does not include a separate dynamics loss term, making it difficult to directly evaluate whether modifications to the dynamics network improve state transition predictions. We can only assess downstream effects through final agent performance metrics.
- 3) **Computational Cost:** Each full experiment requires approximately 15 hours of A100 GPU time, limiting our ability to run extensive hyperparameter sweeps or multiple replicates for statistical validation.
- 4) **Feature Flattening Issues:** The current implementation flattens 4-dimensional input states before feeding them to a single STU layer. This may destroy spatial structure that could be important for learning dynamics.

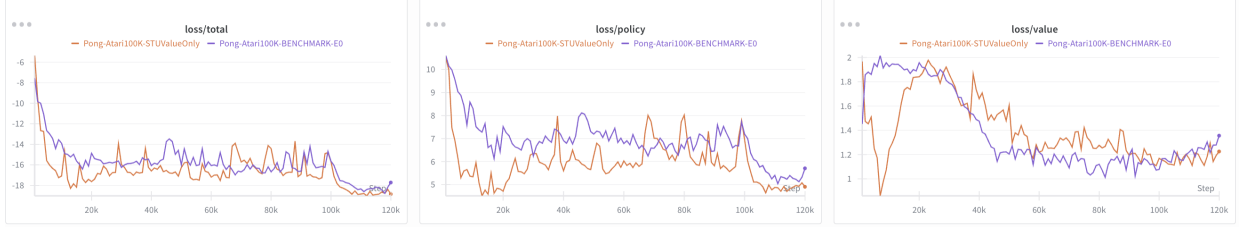


Fig. 1: Experiment 1: STU added to value prediction only.

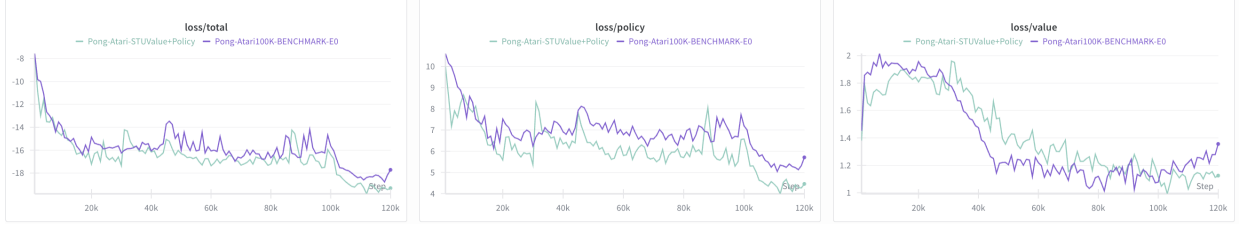
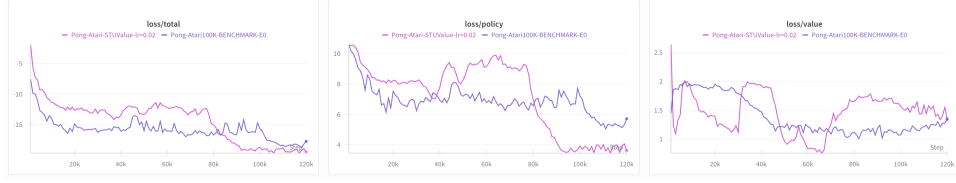
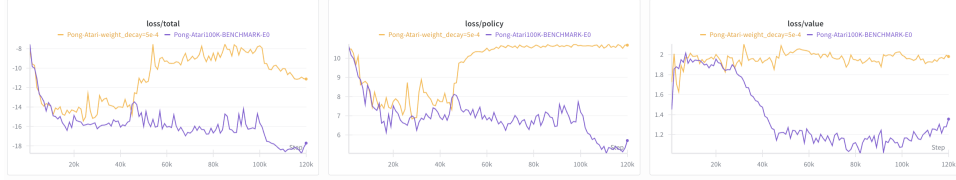


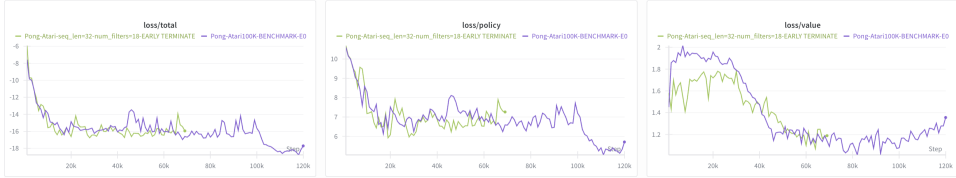
Fig. 2: Experiment 2: STU in value and policy prediction.



(a) Reduced learning rate ($10\times$ decrease)



(b) Increased weight decay ($5\times$ increase)



(c) Increased filters ($8 \rightarrow 16$)

Fig. 3: Experiments 3-5: Hyperparameter tuning attempts.

5) **History Buffer Implementation:** The HistoryMiniSTU approach (Experiment 7) shows promise but requires further investigation into the root cause for unexpected early termination of training loop across several experimental runs. The issue may be related to exceeding memory capacity due to an excessively large input state.

VI. NEXT STEPS

Based on the challenges identified in Section 5, we plan to pursue the following experimental approaches next:

- 1) **Fix History Buffer Implementation:** Resolve the issues with the HistoryMiniSTU implementation that cause early training termination. The history buffer should be created within the training loop to properly track sequential state inputs.
- 2) **Parallel STU Architecture:** Instead of flattening the 4-dimensional input state and feeding it to a single STU, implement 4 parallel STU modules—one for each dimension. This architecture should preserve spatial structure and may improve dynamics learning.
- 3) **Offline Dynamics Network Training:** This is the most

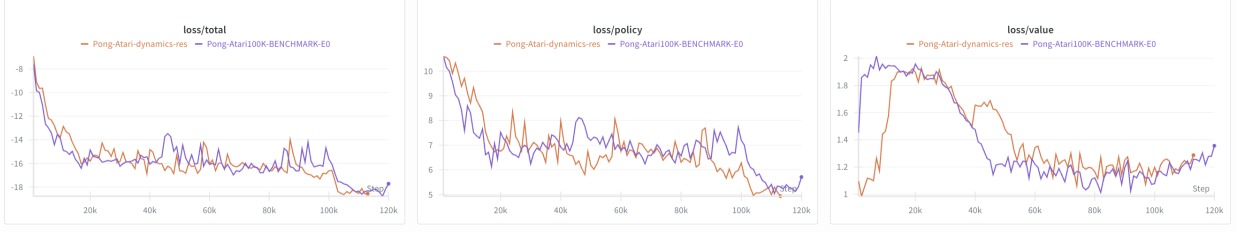


Fig. 4: Experiment 6: STU in dynamics network with residual connection.

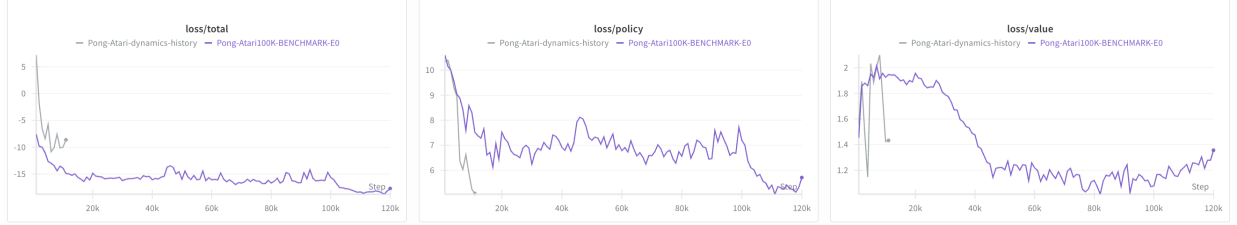


Fig. 5: Experiment 7: STU with history buffer in dynamics network.

promising direction for faster iteration. The approach involves:

- Generate a dataset using a checkpoint of the benchmark model after 100K training steps
- Train an offline encoder-decoder using the learned dynamics network outputs from the benchmark
- Evaluate STU-based modifications to the dynamics network offline before introducing them into the online training loop
- This will dramatically increase iteration speed (no need for 15-hour full training runs) and provide direct metrics for dynamics learning quality through reconstruction loss

4) **Pre-training with Fine-tuning:** If the above experiments do not yield improvements, explore a hybrid approach:

- Train using the benchmark model for N steps (e.g., $N = 10,000$)
- Fine-tune using STU-based modifications
- Evaluate whether STU fine-tuning achieves comparable accuracy to the final benchmark in fewer total steps
- This approach may leverage the stability of standard architectures early in training while benefiting from STU’s temporal modeling capabilities later

VII. CONCLUSION

This midpoint report documents progress toward building world models for reinforcement learning using Spectral Transfer Units. Through seven experiments on the Atari Pong environment, we have systematically explored different integration points for STUs within the EfficientZero architecture, including value prediction, policy prediction, and dynamics networks.

The most encouraging result is from Experiment 1, which demonstrated that STU-augmented value prediction can

achieve target loss values $8.3\times$ faster than the baseline. While this acceleration was followed by instability, it validates the core hypothesis that STUs can efficiently capture temporal structure in RL world models. The challenges encountered—particularly overfitting and the computational cost of full training runs—have motivated our planned offline training methodology.

REFERENCES

- [1] E. Hazan, K. Singh, and C. Zhang, “Learning linear dynamical systems via spectral filtering,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] S. Wang, S. Liu, W. Ye, J. You, and Y. Gao, “Efficientzero v2: Mastering discrete and continuous control with limited data,” *arXiv preprint arXiv:2403.00564*, 2024.
- [3] N. Agarwal, D. Suo, X. Chen, and E. Hazan, “Spectral state space models,” *arXiv preprint arXiv:2312.06837*, 2023.
- [4] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel *et al.*, “Mastering atari, go, chess and shogi by planning with a learned model,” *Nature*, vol. 588, no. 7839, pp. 604–609, 2020.